

LangChain ReAct Agent 项目完整帮助文档

基于当前项目 HELP.md 自动生成

2026-05-06

目录

LangChain ReAct Agent 项目完整帮助文档

本文档基于当前仓库代码、原 HELP.md、STUDY_AND_RESUME_PLAN.md，并参考 LangChain、Chroma、Streamlit、DashScope 官方资料整理。目标是让你可以直接理解这个项目的整体结构、核心知识点、运行方式、扩展方式，以及它在简历中该如何表达。

1 1. 项目一句话说明

这是一个基于 Streamlit + LangChain ReAct Agent + Chroma + DashScope 的扫地机器人智能客服演示项目。

它把用户问题交给 ReAct Agent 判断：如果是产品知识、故障排查、选购建议，就调用 RAG 知识库；如果是个人使用报告，就按固定工具链读取外部用户记录；如果是耗材、维护、保养问题，就生成耗材更换提醒；如果问题涉及天气和环境，还可以调用模拟天气工具辅助回答。

2 2. 项目能做什么

当前项目主要支持四类能力：

1. 知识库问答

从 data/ 目录中的 txt/pdf 文档检索相关内容，再交给大模型总结回答。

1. 工具辅助问答

ReAct Agent 可以根据问题自动选择工具，例如 `get_weather`、`get_user_location`、`get_user_id`、`get_current_month`。

1. 个性化使用报告

Agent 按固定顺序获取用户 ID、月份、报告上下文和外部数据，然后生成个人扫地机器人使用报告。

1. 耗材维护提醒

Agent 可以读取用户在指定月份的耗材记录，生成滤网、边刷、主刷、胶刷、尘盒、水箱等维护建议。

前端入口是 `app.py`。启动后会打开一个 Streamlit 聊天页面，页面标题为“智扫通机器人智能客服”。

3 3. 项目主链路

真实调用链路如下：

```
Streamlit(app.py)
-> ReactAgent(agent/react_agent.py)
-> AgentExecutor.stream()
-> Tools(agent/tools/agent_tools.py)
-> RAG(rag/rag_service.py)
-> Chroma(rag/vector_store.py)
-> DashScope(model/factory.py)
-> Prompt(prompts/*.txt)
```

用户在 app.py 的 st.chat_input() 输入问题后,页面会调用 ReactAgent.execute_stream()。ReactAgent 内部通过 create_react_agent() 创建 ReAct Agent, 再用 AgentExecutor 包装执行。最终答案通过 AgentExecutor.stream() 分块返回给 Streamlit 页面,并由 st.write_stream() 展示。

4 4. 目录结构

```
.
├── app.py                # Streamlit 应用入口
├── requirements.txt      # Python 依赖
├── HELP.md              # 当前完整帮助文档
├── STUDY_AND_RESUME_PLAN.md # 学习与简历沉淀笔记
├── config/              # YAML 配置文件
│   ├── agent.yml        # 外部数据路径配置
│   ├── chroma.yml       # Chroma 与知识库切分配置
│   ├── prompts.yml      # Prompt 文件路径配置
│   └── rag.yml          # 模型名称配置
├── agent/
│   ├── react_agent.py   # ReAct Agent 创建与流式执行逻辑
│   └── tools/
│       ├── agent_tools.py # Agent 可调用工具集合
│       └── middleware.py  # 中间件示例代码
├── rag/
│   ├── rag_service.py   # RAG 检索与总结链
│   └── vector_store.py  # 知识库入库与检索器创建
├── model/
│   └── factory.py        # 聊天模型与 Embedding 模型工厂
├── prompts/             # Agent、RAG、报告生成 Prompt
└── data/                # 知识库文档与外部数据
```

```

扫地机器人 100 问.pdf
扫地机器人 100 问 2.txt
扫拖一体机器人 100 问.txt
故障排除.txt
维护保养.txt
选购指南.txt
external/records.csv      # 用户使用记录
utils/                    # 配置、路径、文件、日志工具
chroma_db/                # Chroma 持久化目录
logs/                     # 运行日志目录

```

5 5. 环境要求与安装

需要准备:

- Python 3.10 或更高版本。当前项目已有 `.venv`。
- 可用的阿里云 DashScope API Key。
- Windows PowerShell、CMD、macOS 或 Linux Shell 均可运行。

安装依赖:

```
pip install -r requirements.txt
```

如果下载较慢, 可使用清华源:

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

当前 `requirements.txt` 固定了主要版本:

```

langchain==0.3.7
langchain-core==0.3.18
langchain-community==0.3.7
langchain-chroma==0.1.4
chromadb==0.5.15
langchain-text-splitters==0.3.2
langgraph==0.2.50
streamlit==1.40.1
pypdf==5.1.0
pyyaml==6.0.2
dashscope==1.20.14

```

6 6. 配置 API Key

项目通过 DashScope 调用通义千问聊天模型和 Embedding 模型,运行前需要配置 DASHSCOPE_API_KEY。

PowerShell 临时配置:

```
$env:DASHSCOPE_API_KEY="your-api-key"
```

CMD 临时配置:

```
set DASHSCOPE_API_KEY=your-api-key
```

macOS/Linux 临时配置:

```
export DASHSCOPE_API_KEY="your-api-key"
```

临时环境变量只对当前终端窗口有效。关闭终端后需要重新配置。

7 7. 核心配置说明

7.1 7.1 模型配置

文件: config/rag.yml

```
chat_model_name: qwen3-max  
embedding_model_name: text-embedding-v4
```

- chat_model_name: 聊天模型名称, 当前使用 qwen3-max。
- embedding_model_name: 向量化模型名称, 当前使用 text-embedding-v4。

这两个配置会在 model/factory.py 中读取:

```
chat_model = ChatTongyi(model=rag_conf["chat_model_name"])  
embed_model = DashScopeEmbeddings(model=rag_conf["embedding_model_name"])
```

7.2 7.2 向量库配置

文件: config/chroma.yml

```
collection_name : agent
persist_directory : chroma_db
k : 3
data_path : data
md5_hex_store : md5.txt
allow_knowledge_file_type : ["txt","pdf"]
chunk_size : 200
chunk_overlap : 20
separators : ["\n\n","。",".", "?", "? ", "!", " ", ""]
```

字段解释:

- collection_name: Chroma 集合名称。
- persist_directory: 向量库持久化目录。
- k: 每次检索返回的 top-k 文档片段数量。
- data_path: 知识库文档所在目录。
- md5_hex_store: 记录已入库文件 MD5 的文件, 用来避免重复入库。
- allow_knowledge_file_type: 允许入库的文件类型。
- chunk_size: 每个文本块最大长度。
- chunk_overlap: 相邻文本块重叠长度。
- separators: 文本切分时优先使用的分隔符。

7.3 7.3 Prompt 配置

文件: config/prompts.yml

```
main_prompt_path : prompts/main_prompt.txt
rag_summarize_prompt_path : prompts/rag_summarize.txt
report_prompt_path : prompts/report_prompt.txt
```

- main_prompt.txt: Agent 主提示词, 定义角色、工具能力边界、报告生成强约束。
- rag_summarize.txt: RAG 检索后总结使用的提示词, 要求答案基于参考资料。
- report_prompt.txt: 报告生成场景提示词。

7.4 7.4 外部数据配置

文件: config/agent.yml

```
external_data_path: data/external/records.csv
```

records.csv 模拟业务系统中的用户使用记录。字段包括:

用户 ID, 特征, 清洁效率, 耗材, 对比, 时间

报告工具和耗材提醒工具会按 用户 ID, 月份查询对应记录。

8 8. 核心知识点直接答案

8.1 8.1 什么是 RAG

RAG 是 Retrieval-Augmented Generation, 即“检索增强生成”。它先从外部知识库检索和问题相关的资料, 再把这些资料连同用户问题一起交给大模型生成答案。

本项目中的 RAG 是:

用户问题 -> Chroma 检索相关文档 -> 拼接 context -> DashScope 模型总结回答

它解决的问题是: 大模型自身不知道本地文档里的项目资料、产品问答和故障排查内容, 所以需要先检索, 再回答。

8.2 8.2 为什么要把文档切成 chunk

因为原始文档可能很长, 不能整篇都塞给 Embedding 或聊天模型。切成 chunk 后, 每个小片段都可以单独向量化和检索。

切分的好处:

- 提高检索粒度: 只找最相关的片段。
- 降低上下文长度: 减少传给模型的无关内容。
- 提升命中率: 同一篇文档中不同主题可以分别被检索到。

8.3 8.3 chunk_size、chunk_overlap、k 分别影响什么

- chunk_size 控制每个文本块的最大长度。太小容易割裂上下文, 太大容易混入无关信息。
- chunk_overlap 控制相邻文本块的重叠长度。适当重叠能减少断句导致的信息丢失, 但过大容易重复入库。
- k 控制每次检索返回多少个片段。太小可能漏掉答案, 太大可能让模型看到噪声。

当前配置是 chunk_size=200、chunk_overlap=20、k=3, 适合这个中小型中文知识库的演示场景。

8.4 8.4 什么是 Embedding

Embedding 是把文本转换成向量数字的过程。语义相近的文本，向量距离通常更近。

本项目使用 `DashScopeEmbeddings(model="text-embedding-v4")` 把知识库片段转成向量，然后存入 Chroma。用户提问时，也会被转成向量，再和库里的向量做相似度检索。

8.5 8.5 什么是 Chroma

Chroma 是一个向量数据库，用来存储文档、向量和元数据，并支持相似度查询。

本项目用 `langchain_chroma.Chroma` 创建本地向量库：

```
Chroma(  
    collection_name="agent",  
    embedding_function=embed_model,  
    persist_directory="chroma_db",  
)
```

`persist_directory="chroma_db"` 表示向量库会持久化到本地目录，下次启动仍可复用。

8.6 8.6 什么是 Retriever

Retriever 是检索器。它把“向量数据库怎么查”封装成一个统一接口。

本项目中的代码是：

```
self.vector_store.as_retriever(search_kwargs={"k": chroma_conf["k"]})
```

调用 `retriever.invoke(query)` 后，会返回和 `query` 最相关的 `Document` 列表。

8.7 8.7 什么是 Agent

Agent 是能“自己决定下一步做什么”的大模型应用。普通 LLM 调用只生成回答，Agent 则可以根据问题选择工具、调用工具、观察结果，再决定是否继续调用工具或输出最终答案。

一句话区分：

普通 RAG：先检索，再回答。

Agent：模型自己判断是否检索、调用哪个工具、调用几次。

8.8 8.8 什么是 ReAct

ReAct 是 Reasoning + Acting，即“推理 + 行动”。模型按如下格式工作：

Thought -> Action -> Action Input -> Observation -> Final Answer

在本项目中：

- Thought：模型分析用户需求。
- Action：选择工具名，例如 `rag_summarize`。
- Action Input：传给工具的参数。
- Observation：工具返回结果。
- Final Answer：整合结果后给用户的最终回答。

8.9 8.9 create_react_agent() 做什么

`create_react_agent()` 把聊天模型、工具列表和 ReAct Prompt 组合成一个可运行的 Agent。

本项目在 `agent/react_agent.py` 中调用：

```
agent = create_react_agent(  
    llm=chat_model,  
    tools=tools,  
    prompt=prompt  
)
```

注意：ReAct Prompt 必须包含 `tools`、`tool_names`、`agent_scratchpad` 等变量，否则 Agent 无法知道有哪些工具，也无法记录中间推理和工具观察。

8.10 8.10 AgentExecutor 做什么

AgentExecutor 负责真正执行 Agent 循环，包括：

- 把用户输入交给 Agent。
- 解析模型输出中的 Action 和 Action Input。
- 调用对应工具。
- 把工具结果作为 Observation 交回模型。
- 循环直到模型输出 Final Answer。

本项目还配置了 `handle_parsing_errors`，当模型输出格式不符合 ReAct 要求时，给模型一个纠错提示，减少前端崩溃。

8.11 8.11 @tool 装饰器有什么用

@tool 会把普通 Python 函数包装成 LangChain Tool。函数名会成为工具名，函数 docstring 会成为工具描述，模型会根据工具描述判断什么时候调用它。

例如：

```
@tool
def get_weather(city: str) -> str:
    """ 获取指定城市的天气，以消息字符串的形式返回"""
    return "..."
```

所以新增工具时，docstring 要写清楚“工具能做什么、什么时候用、入参格式是什么”。

8.12 8.12 Streamlit 在项目中负责什么

Streamlit 负责前端聊天界面：

- st.title() 显示页面标题。
- st.chat_input() 接收用户输入。
- st.chat_message() 展示用户和助手消息。
- st.session_state 保存聊天历史和 Agent 实例。
- st.write_stream() 流式展示模型答案。

8.13 8.13 为什么要用 YAML 配置

YAML 把模型名、Prompt 路径、Chroma 参数、外部数据路径从代码中抽离出来。这样调整模型、切分参数、知识库路径时不需要改核心代码。

当前配置分工：

- rag.yml 管模型。
- chroma.yml 管向量库和知识库切分。
- prompts.yml 管 Prompt 文件路径。
- agent.yml 管外部业务数据路径。

9 9. 知识库构建与更新

如果 chroma_db/ 中已有可用向量库，可以直接运行应用。

如果新增或修改了 data/ 中的知识库文档，需要重新执行入库逻辑：

```
python -m rag.vector_store
```

入库流程在 `rag/vector_store.py`:

1. 扫描 `config/chroma.yml` 中 `data_path` 指向的目录。
2. 只读取 `txt/pdf` 文件。
3. 计算文件 MD5, 跳过已经处理过的文件。
4. 用 `TextLoader` 或 `PyPDFLoader` 加载文档。
5. 用 `RecursiveCharacterTextSplitter` 切分文档。
6. 用 `DashScope Embedding` 生成向量。
7. 写入 `Chroma`。
8. 将已处理文件 MD5 写入 `md5.txt`。

注意: 如果想完全重建向量库, 需要手动确认旧的 `chroma_db/` 和 `md5.txt`。不要在脚本中批量删除目录。

10 10. 启动应用

在项目根目录执行:

```
streamlit run app.py
```

启动成功后, 终端通常显示:

Local URL: `http://localhost:8501`

打开浏览器访问该地址即可使用聊天界面。

11 11. 推荐测试问题

知识库问答:

扫地机器人有哪些主要功能?

故障排查:

机器人无法正常回充, 应该怎么办?

选购建议:

小户型适合选择什么类型的扫地机器人?

使用报告：

请给我生成一份本月扫地机器人使用报告

耗材维护：

帮我看看这个月滤网和边刷需不需要更换

天气环境：

帮我看看深圳今天的天气是否适合拖地

12 12. 工作流程详解

12.1 12.1 普通知识库问答

用户输入产品问题

- > Streamlit 调用 `ReactAgent.execute_stream()`
- > Agent 判断需要专业资料
- > 调用 `rag_summarize`
- > `RagSummarizeService` 检索 Chroma
- > 拼接参考资料 `context`
- > `DashScope` 模型基于参考资料生成回答
- > Streamlit 流式展示答案

12.2 12.2 使用报告生成

报告生成在 `agent/react_agent.py` 的 Prompt 中被强约束为固定顺序：

1. 调用 `get_user_id` 获取用户 ID。
2. 调用 `get_current_month` 获取月份。
3. 调用 `fill_context_for_report` 注入报告场景上下文。
4. 调用 `fetch_external_data`，传入 " 用户 ID，月份"。
5. 根据外部数据生成最终报告。

当前 `get_user_id` 和 `get_current_month` 使用随机示例数据，所以每次生成报告的用户和月份可能不同。

12.3 12.3 耗材维护提醒

耗材维护提醒使用 `generate_consumable_reminder(params_str)` 工具。

调用格式：

用户 ID, 月份

示例:

1001, 2025-03

如果用户没有明确给出用户 ID 或月份, Agent 会先调用 `get_user_id` 和 `get_current_month`, 再把结果拼成 " 用户 ID, 月份" 传入工具。

工具会返回:

- 用户画像。
- 清洁表现。
- 耗材状态。
- 同类对比。
- 维护建议。

13 13. 主要代码入口

13.1 13.1 app.py

负责 Streamlit 页面:

- 初始化 `ReactAgent`。
- 保存聊天历史到 `st.session_state["message"]`。
- 接收用户输入。
- 调用 `execute_stream()` 获取流式响应。
- 用 `st.write_stream()` 展示答案。

注意点: 当前代码最后保存助手消息时使用 `response_messages[-1]`, 这通常只保存最后一个流式 chunk。如果以后要完整保存整段回复, 可改为 `"".join(response_messages)`。

13.2 13.2 agent/react_agent.py

负责构建 ReAct Agent:

- 加载主 Prompt。
- 注册工具列表。
- 拼接 ReAct 标准模板。
- 使用 `create_react_agent()` 创建 Agent。
- 使用 `AgentExecutor` 包装 Agent。
- 通过 `execute_stream()` 提供流式输出。

当前工具列表：

```
tools = [  
    rag_summarize,  
    get_weather,  
    get_user_location,  
    get_user_id,  
    get_current_month,  
    fetch_external_data,  
    fill_context_for_report,  
    generate_consumable_reminder,  
]
```

13.3 13.3 agent/tools/agent_tools.py

定义 Agent 可调用工具：

- rag_summarize: 知识库检索问答。
- get_weather: 返回模拟天气。
- get_user_location: 随机返回用户城市。
- get_user_id: 随机返回用户 ID。
- get_current_month: 随机返回月份。
- fetch_external_data: 读取 records.csv 中的外部记录。
- generate_consumable_reminder: 生成耗材维护提醒。
- fill_context_for_report: 报告场景占位工具。

13.4 13.4 rag/vector_store.py

负责知识库入库和检索器创建：

- 初始化 Chroma。
- 扫描知识库目录。
- 加载 txt/pdf 文件。
- 切分文档。
- 写入向量库。
- 返回 retriever。

13.5 13.5 rag/rag_service.py

负责 RAG 问答链：

- 调用 retriever 检索相关文档。

- 拼接参考资料上下文。
- 使用 `rag_summarize.txt` 限制模型基于资料回答。
- 用 `StrOutputParser()` 输出纯字符串。

13.6 13.6 model/factory.py

负责模型初始化：

- ChatTongyi：聊天模型。
- DashScopeEmbeddings：Embedding 模型。

14 14. 如何新增知识库文档

1. 将 `.txt` 或 `.pdf` 文件放入 `data/` 目录。
2. 确认 `config/chroma.yml` 中允许该文件类型。
3. 执行：

```
python -m rag.vector_store
```

1. 启动应用并提问验证。

建议：

- `txt` 文件使用 UTF-8 编码。
- 文档内容按主题分段。
- 问题经常检索不到时，可适当调大 `k`，或优化知识库文本表达。
- 如果文档改动后没有重新入库，检查 `md5.txt` 和日志。

15 15. 如何新增 Agent 工具

新增工具通常改两处：`agent/tools/agent_tools.py` 和 `agent/react_agent.py`。

第一步，在 `agent/tools/agent_tools.py` 中定义工具：

```
from langchain_core.tools import tool

@tool
def get_order_status(order_id: str) -> str:
    """ 根据订单号查询订单状态。调用时传入订单号字符串。"""
    return " 订单已完成"
```

第二步，在 `agent/react_agent.py` 中导入并加入工具列表：

```
tools = [  
    rag_summarize,  
    get_weather,  
    get_order_status,  
]
```

第三步，在 Prompt 中说明工具调用场景和入参格式。工具 docstring 和 Prompt 说明越清楚，Agent 越容易正确调用。

16 16. 常见问题

16.1 16.1 启动时报 DashScope API Key 错误

检查是否设置了环境变量：

```
$env:DASHSCOPE_API_KEY
```

如果没有输出，重新设置：

```
$env:DASHSCOPE_API_KEY="your-api-key"
```

16.2 16.2 问答结果和知识库内容无关

检查：

- chroma_db/ 是否已经生成。
- 是否执行过 `python -m rag.vector_store`。
- data/ 中是否有相关文档。
- config/chroma.yml 中 k 是否太小。
- 文档编码是否为 UTF-8。
- rag_summarize.txt 是否仍要求模型基于参考资料回答。

16.3 16.3 新增文档后没有被检索到

项目用 MD5 记录已处理文件。如果文档已经入库，再次运行时会跳过相同 MD5 的文件。可以查看 logs/ 中的日志确认是否跳过。

如果确实需要完全重建，请手动确认后处理旧向量库和 md5.txt，不要使用批量删除命令。

16.4 16.4 Agent 输出格式错误或中断

ReAct Agent 对输出格式要求较严格，模型需要生成：

Thought / Action / Action Input / Observation / Final Answer

项目已在 AgentExecutor 中配置 `handle_parsing_errors`。如果仍失败，可以：

- 换一种更明确的问题问法。
- 检查 `prompts/main_prompt.txt` 是否被改坏。
- 换用更稳定的聊天模型。
- 减少 Prompt 中和 ReAct 标准格式冲突的说明。

16.5 16.5 Streamlit 页面聊天历史不完整

当前 `app.py` 保存助手回复时使用 `response_messages[-1]`，如果流式输出被切成多个 chunk，历史里可能只保存最后一段。可改为：

```
st.session_state["message"].append({
    "role": "assistant",
    "content": "".join(response_messages),
})
```

16.6 16.6 README 或终端显示乱码

中文文档在部分 Windows 终端中可能因编码显示异常。建议使用支持 UTF-8 的编辑器查看 Markdown，并优先参考本 `HELP.md`。

17 17. 学习路线

建议按以下顺序学习：

1. 先看 `app.py`，理解前端怎么接收问题和展示答案。
2. 再看 `agent/react_agent.py`，理解 Agent 怎么创建、怎么注册工具。
3. 再看 `agent/tools/agent_tools.py`，理解每个工具具体做什么。
4. 再看 `rag/vector_store.py`，理解文档如何入库。
5. 再看 `rag/rag_service.py`，理解检索结果如何交给模型总结。
6. 最后看 `prompts/*.txt` 和 `config/*.yaml`，理解配置和提示词如何影响效果。

18 18. 简历写法

可以写成：

基于 LangChain + ReAct Agent + Chroma 实现智能客服问答系统，支持扫地机器人知识库问答、工具调用、个性化使用报告和耗材维护提醒生成。项目使用 DashScope 模型完成对话与 Embedding，基于 Chroma 构建本地向量知识库，通过 ReAct Agent 动态选择 RAG 检索、用户数据查询、报告生成和耗材提醒等工具，并使用 Streamlit 实现流式对话界面。

可量化亮点：

- 设计并实现 RAG 检索问答链路，支持 PDF/TXT 文档入库、文本切分、向量化存储和 top-k 检索。
- 基于 LangChain ReAct Agent 注册多个工具，实现知识库问答、用户数据查询、报告生成和耗材维护提醒等多任务调度。
- 使用 YAML 管理模型、Prompt、Chroma 和 Agent 配置，降低系统扩展成本。
- 构建 Streamlit 聊天界面，支持用户连续对话和流式输出。
- 增加耗材维护提醒工具，将结构化业务数据接入 Agent 决策流程。

19 19. 验收问题与答案

19.1 RAG 为什么需要把文档切成 chunk

因为文档通常很长，直接整体检索粒度太粗，也容易超过模型上下文。chunk 可以让系统按小片段做向量检索，提高相关性并减少噪声。

19.2 chunk_size、chunk_overlap、k 分别影响什么

chunk_size 影响片段长度，chunk_overlap 影响相邻片段的上下文连续性，k 影响每次检索返回多少片段。

19.3 rag_summarize 什么时候会被 Agent 调用

当用户询问扫地机器人知识库问题、故障排查、选购建议、使用方法、维护保养等专业问题时，Agent 应优先调用 rag_summarize。

19.4 AgentExecutor 和普通 LLM 调用有什么区别

普通 LLM 调用只输入问题、输出回答。AgentExecutor 会执行 Agent 循环，支持模型选择工具、调用工具、读取观察结果并继续推理。

19.5 新增一个工具需要改哪些地方

通常需要：

1. 在 agent/tools/agent_tools.py 中用 @tool 定义函数。
2. 在 agent/react_agent.py 中导入并加入 tools 列表。
3. 在 Prompt 中补充调用场景和入参格式。

20 20. 一键运行流程

首次运行：

```
cd D:\my_develop\A_work_program\LangChain-ReAct-Agent-main
pip install -r requirements.txt
$env:DASHSCOPE_API_KEY="your-api-key"
python -m rag.vector_store
streamlit run app.py
```

后续运行：

```
cd D:\my_develop\A_work_program\LangChain-ReAct-Agent-main
$env:DASHSCOPE_API_KEY="your-api-key"
streamlit run app.py
```

21 21. 参考资料

- LangChain ReAct Agent API:<https://reference.langchain.com/python/langchain-classic/a>
- LangChain AgentExecutor API:<https://reference.langchain.com/python/langchain-classic/a>
- LangChain Tools 文档:<https://docs.langchain.com/oss/python/langchain-tools>
- LangChain RecursiveCharacterTextSplitter 文档:<https://reference.langchain.com/python/la>
- LangChain Chroma 集成文档:<https://docs.langchain.com/oss/python/integrations/vector>
- Chroma 官方 Getting Started:<https://docs.trychroma.com/docs/overview/getting-started>
- Chroma Collections 文档:<https://docs.trychroma.com/docs/collections/manage-collection>
- Streamlit Chat Elements 文档:<https://docs.streamlit.io/library/api-reference/chat>

- Streamlit `st.chat_input` 文档:https://docs.streamlit.io/develop/api-reference/chat/st.chat_input
- LangChain ChatTongyi 文档:<https://docs.langchain.com/oss/python/integrations/chat/tongyi/>
- LangChain DashScope Embeddings 文档:<https://docs.langchain.com/oss/python/integrations/embeddings/dashscope/>
- 阿里云 DashScope Embedding 文档:<https://www.alibabacloud.com/help/en/model-studio/de>